



Security Audit Report

Kaskad TEE Oracle (Nuntius)

Enclave Sealing, Aggregation Policy, On-Chain Freshness & Relay Path

Version: 1.0
Date: July 2026
Auditor: The Sage Audit
Classification: Confidential
Scope: kaskad-nuntius (enclave, contracts, relayer, infra)

This report is confidential and intended solely for the

Client.

Unauthorized distribution, copying, or disclosure is strictly prohibited.

Contents

1	Introduction	2
1.1	Disclaimer	2
1.2	Security Assessment Methodology	2
1.3	Finding Severity Breakdown	3
1.4	Project Overview	3
1.5	Findings Summary	3
2	Findings	4
2.1	Host-Controlled Sealing Route	4
2.2	Equal-Weight Fallback Remains Publishable	6
2.3	Circuit Breaker vs Withdraw Freshness	8
2.4	Serial Source Fetching	10
2.5	Relayer Quorum Mismatch	12
3	Appendix	13
3.1	PoC Artifact Index	13

1 Introduction

1.1 Disclaimer

The audit makes no statements or warranties about utility of the code, safety of the code, suitability of the business model, investment advice, endorsement of the platform or its products, regulatory regime for the business model, or any other statements about fitness of the contracts to purpose, or their bug free status. The audit documentation is for discussion purposes only.

The information presented in this report is confidential and privileged. If you are reading this report, you agree to keep it confidential, not to copy, disclose or disseminate without the agreement of the Client. If you are not the intended recipient(s) of this document, please note that any disclosure, copying or dissemination of its content is strictly forbidden.

1.2 Security Assessment Methodology

A group of auditors are involved in the work on the audit. Security engineers check the provided source code independently of each other in accordance with the methodology described below:

1.2.0 1. Project Architecture Review

- Project documentation review.
- General code review.
- Reverse research and study of the project architecture on the source code alone.

Stage goals

- Build an independent view of the project's architecture.
- Identifying logical flaws.

1.2.0 2. Checking the Code Against the Vulnerabilities Checklist

- Manual code check for vulnerabilities listed on the Contractor's internal checklist.
- Code check with the use of static analyzers where applicable.

Stage goal

Eliminate typical vulnerabilities and trust-boundary failures in TEE-backed oracles.

1.2.0 3. Checking the Code for Compliance with the Desired Security Model

- Detailed study of the project documentation and sealing / attestation comments.
- Examination of contracts tests and enclave PoCs.
- Comparison of the desired host-untrusted Nitro model with the live control flow.
- Exploit PoC development (Foundry / Cargo).

1.2.0 4. Consolidation of the Auditors' Interim Reports

- Cross check and discussion of issues.
- Issuance of an interim audit report.

1.3 Finding Severity Breakdown

Severity	Description
Critical	Vulnerabilities that can result in significant loss of funds, irreversible contract breakage, or complete protocol compromise. Immediate fixing is required.
High	Vulnerabilities that can lead to loss of funds, incorrect behavior under common conditions, or significant protocol dysfunction. Fixing is strongly recommended.
Medium	Vulnerabilities that can cause undesirable behavior under specific conditions or degrade user experience. Fixing is recommended.
Low	Minor issues, deviations from best practices, or code quality concerns. Fixing is suggested.
Informational	Observations that do not represent an immediate security risk but may improve clarity or maintainability.

1.4 Project Overview

Kaskad Nuntius is a Nitro TEE price oracle. The enclave fetches exchange quotes, aggregates a median, and EIP-191-signs price updates. On-chain components (`KaskadPriceOracle`, `KaskadAggregatorV3`, `KaskadRouter`) accept those updates and serve Aave-style consumers. A TypeScript relayer pushes signed prices into the on-chain cache; users can also pull fresh signatures through the router.

This assessment focused on: (1) host $\leftrightarrow$ enclave trust boundaries around key sealing, (2) aggregation fail-open policy, (3) on-chain freshness / circuit-breaker interaction with lending exits, and (4) relay and fetch-path liveness against the documented freshness window.

1.5 Findings Summary

ID	Severity	Title
C-01	Critical	Host-controlled sealing route redirects first-boot key sealing
H-01	High	Equal-weight fallback remains publishable (TIBBIR instance + general policy)
H-02	High	Circuit breaker blocks crash updates while withdraw skips freshness
M-01	Medium	Serial source fetching accumulates timeouts past router freshness
M-02	Medium	Relayer hardcodes <code>MIN_SOURCES = 3</code> and drops valid two-source updates

2 Findings

2.1 Host-Controlled Sealing Route Lets a Malicious Parent Instance Redirect First-Boot Key Sealing

ID: C-01
Severity: **Critical**
Components: `src/aws_creds.rs`, `src/sealing.rs`, `src/signer.rs`,
`enclave/aws-creds-proxy.py`, `infra/modules/oracle/user-data-prod.sh`

2.1.0 Links to Root Cause

- Host VSOCK blob includes `Region`, `EifBucket`, `KmsSealingAlias`: `src/aws_creds.rs`
- Proxy injects those fields from host env: `enclave/aws-creds-proxy.py`
- First-boot `seal_and_upload` uses host-supplied `KeyId` and S3 host: `src/sealing.rs`
- First boot generates NSM seed then seals: `src/signer.rs`

2.1.0 Impact

Full oracle signer compromise under the repository’s stated enclave trust model (parent host outside the trusted boundary for randomness, attestation, and transport safety).

A malicious or compromised parent instance can steer first-boot sealing to attacker-owned AWS resources, recover the oracle seed outside the enclave, and forge arbitrary signed price updates that downstream consumers accept as authentic. The compromise survives beyond the initial host access because the attacker owns the long-lived signer material.

2.1.0 Description

The sealing design is intended to persist the enclave `secp256k1` key across spot reclaims without exposing plaintext to the host. First boot generates an NSM-rooted seed and calls `seal_and_upload`.

However, the inputs that determine *where* sealing happens are not measured or authenticated inside the enclave. They are supplied by the parent host over VSOCK:

- `aws_creds.rs` documents and deserializes host JSON containing `Region`, `EifBucket`, and `KmsSealingAlias`.
- `user-data-prod.sh` sets `KASKAD_AWS_REGION`, `KASKAD_EIF_BUCKET`, and `KASKAD_KMS_SEALING_ALIAS`.
- `aws-creds-proxy.py` merges those environment variables into the JSON returned to the enclave.

The sealing path then uses those host-chosen values directly:

- `creds.kms_sealing_alias` as `KeyId` for `kms:Encrypt`
- `creds.region` for the KMS endpoint
- `creds.eif_bucket` / `creds.region` for S3 GET/PUT of `sealed-key.bin`

Exploit path on first boot:

1. Parent returns attacker-controlled credentials plus attacker-controlled region, bucket, and KMS alias.

2. Enclave generates a fresh NSM-rooted seed.
3. `seal_and_upload()` encrypts that seed under the attacker-selected KMS key.
4. Enclave uploads ciphertext to the attacker-selected S3 bucket.
5. Attacker decrypts with its own KMS key outside the enclave.
6. Attacker owns the long-lived oracle signing seed.

This is not blocked by the decrypt-side attestation policy: the exploit targets first-boot `kms:Encrypt`, not enclave-only `kms:Decrypt` recovery.

2.1.0 Proof of Concept (PoC)

The enclave accepts host-supplied routing fields and constructs KMS/S3 requests to attacker-selected AWS resources:

```

1 #[test]
2 fn host_controlled_sealing_route_can_be_redirected_to_attacker_aws_resources() {
3     // Assert production sources still wire Region / EifBucket / KmsSealingAlias
4     // from the host VSOCK blob into sealing.rs Encrypt + S3 hosts.
5
6     let malicious = HostSuppliedCreds {
7         region: "eu-west-1".into(),
8         eif_bucket: "attacker-sealed-key-bucket".into(),
9         kms_sealing_alias: "alias/attacker-controlled-sealing-key".into(),
10    };
11
12    let kms_host = format!("kms.{}.amazonaws.com", malicious.region);
13    let s3_host = format!(
14        "{}.s3.{}.amazonaws.com",
15        malicious.eif_bucket, malicious.region
16    );
17    let encrypt_body = json!({
18        "KeyId": malicious.kms_sealing_alias,
19        "Plaintext": base64::encode([0x41; 32]),
20    });
21
22    assert_eq!(kms_host, "kms.eu-west-1.amazonaws.com");
23    assert_eq!(s3_host,
24        "attacker-sealed-key-bucket.s3.eu-west-1.amazonaws.com");
25    assert!(encrypt_body.contains("alias/attacker-controlled-sealing-key"));
26 }

```

```

1 cargo test --test exploitability_pocs \
2     host_controlled_sealing_route_can_be_redirected_to_attacker_aws_resources \
3     -- --nocapture

```

2.1.0 Mitigation

- Remove Region, EifBucket, and KmsSealingAlias from the unauthenticated host VSOCK blob.
- Bake the sealing destination into PCR-measured config, or verify it against an enclave-measured allowlist.
- Fail closed if the sealing route does not match the expected deployment identity.

2.2 Equal-Weight Fallback Remains Publishable; TIBBIR/USD Deterministically Selects the Lower Quote

ID: H-01
Severity: **High**
Components: `src/aggregator/mod.rs`, `src/main.rs`,
`src/sources/mexc.rs`, `src/sources/coingecko.rs`,
`config/assets.json`

2.2.0 Links to Root Cause

- Aggregator documents that strict callers should refuse `EqualFallback`: `src/aggregator/mod.rs`
- Publish loop binds mode then discards it (`_mode`): `src/main.rs`
- TIBBIR/USD configured with exactly two sources, both zero-volume adapters: `config/assets.json`, `mexc.rs`, `coingecko.rs`

2.2.0 Impact

The aggregator surfaces `EqualFallback` so callers can refuse to publish. The enclave ignores that signal and still signs. For TIBBIR/USD, the configured sources always force equal-weight fallback, and the two-source case deterministically selects the lower quote — a lower-quoting source pushes the feed down on every cycle. The same fail-open path applies to any asset that enters equal-weight fallback: zero-volume samples get equal votes and can skew the signed median without a volume-weighted majority. Lending consumers that read the feed then misprice collateral and debt.

2.2.0 Description

`weighted_median()` counts how many sources have `volume > 0.0`. If fewer than half do, every source gets weight 1.0 and the function returns `WeightingMode::EqualFallback`. The aggregator documents that strict callers should refuse to publish in that mode.

The main loop discards the mode and publishes anyway:

```
1 let (median, _mode) = match aggregator::weighted_median(&prices) {
2     Some(m) => m,
3     None => { /* skip */ continue; }
4 };
5 // median is cached for signing -- mode is never consulted
```

There is no per-asset policy that can refuse `EqualFallback`.

TIBBIR/USD is a permanent instance of this failure mode:

- `config/assets.json`: `min_sources = 2`, sources `mexc` + `coingecko` only.
- Both adapters hardcode `volume: 0.0`, so every successful cycle is `EqualFallback`.
- With two equal weights, total weight is 2 and half is 1 — the first sorted sample wins, so the lower quote is selected every time.

2.2.0 Proof of Concept (PoC)

A — TIBBIR/USD

```
1 #[test]
2 fn poc_tibbir_equal_fallback_lower_quote_wins() {
3     let cfg = load_assets().expect("embedded assets.json");
4     let asset = cfg.assets.iter()
```

```

5     .find(|a| a.symbol == "TIBBIR/USD").unwrap();
6     assert_eq!(asset.min_sources, 2);
7     // mexc + coingecko only; both emit volume 0.0 in adapters
8
9     let prices = vec![
10         pp(0.91, 0.0, "coingecko", 1_710_000_000),
11         pp(0.87, 0.0, "mexc", 1_710_000_001),
12     ];
13     let (median, mode) = weighted_median(&prices).unwrap();
14     assert_eq!(mode, WeightingMode::EqualFallback);
15     assert_eq!(median, 0.87); // lower sorted quote wins
16
17     let (published, _mode) = (median, mode); // mirrors main.rs
18     assert_eq!(published, 0.87);
19 }

```

B — Equal-fallback skews the publishable median

```

1 #[test]
2 fn POC_equal_fallback_still_produces_publishable_skewed_median() {
3     // 3 honest high-volume ~2000; 4 zero-volume at 2100 -> EqualFallback
4     let prices = vec![
5         pp(2000.0, 10_000.0, "binance"),
6         pp(2001.0, 9_000.0, "okx"),
7         pp(1999.0, 8_000.0, "bybit"),
8         pp(2100.0, 0.0, "atk_a"),
9         pp(2100.0, 0.0, "atk_b"),
10        pp(2100.0, 0.0, "atk_c"),
11        pp(2100.0, 0.0, "atk_d"),
12    ];
13    let (median, mode) = weighted_median(&prices).unwrap();
14    assert_eq!(mode, WeightingMode::EqualFallback);
15    // equal-weight median of sorted set -> 2100 (not ~2000)
16    assert!((median - 2100.0).abs() < 1e-9);
17 }

```

```

1 cargo test --test poc_volume_weight_gap -- --nocapture
2 # and / or TIBBIR-focused test in fail_open_oracle_poc / equivalent

```

2.2.0 Mitigation

- Treat `WeightingMode::EqualFallback` as a hard refusal for assets not explicitly approved for fallback.
- Add `require_volume_weight` (or equivalent) on `AssetConfig`; enable for critical assets.
- Do not configure two-source assets entirely from adapters that always emit `volume: 0.0`.
- Add a regression test that fails if a strict asset publishes after `EqualFallback`.

2.3 Circuit Breaker Blocks Crash Updates While Withdraw Skips Freshness Checks

ID: H-02

Severity: **High**

Components: KaskadPriceOracle.sol, KaskadRouter.sol

2.3.0 Links to Root Cause

- Per-update circuit breaker 15% / 30% after 1h silence: KaskadPriceOracle._checkFreshnessAndBreaker
- withdrawWithPrices intentionally has no freshness gate and accepts an empty bundle: KaskadRouter.sol

2.3.0 Impact

During large downward moves, honest crash updates revert on-chain while the feed remains stale-high. Borrowers can exit collateral through `withdrawWithPrices` (including with an empty price array) against that inflated mark. Protocol bad debt and lost liquidation opportunity follow from the lag window.

2.3.0 Description

The oracle caps each price move at 15% (30% after one hour of on-chain silence). A -40% crash cannot land in a single update, so the on-chain feed remains above the true market.

```
1 uint16 limit = MAX_PRICE_CHANGE_BPS; // 1500 = 15%
2 if (block.timestamp - current.timestamp >= CIRCUIT_BREAKER_STALENESS) {
3     limit = MAX_RESUME_CHANGE_BPS; // 3000 = 30%
4 }
```

`withdrawWithPrices` deliberately does not require asset freshness:

```
1 /// @notice Withdraw 'asset'. No freshness gate -- user exit must not be blocked.
2 function withdrawWithPrices(PriceUpdate[] calldata prices, address asset, uint256 amount)
3     external nonReentrant withSender(msg.sender)
4 {
5     _pushPrices(prices);
6     // transfer aToken, pool.withdraw -- no _requireFreshAsset
7 }
```

Combined effect:

1. Market crashes more than the circuit-breaker cap.
2. Honest signed updates matching the crash revert.
3. Aggregator / oracle price stays stale-high.
4. Borrower calls `withdrawWithPrices([])` and exits against an inflated health factor.

The missing withdraw freshness check is intentional for liveness. Paired with a hard per-update price cap, it creates a window where risky exits remain possible while risk checks still see the old price.

2.3.0 Proof of Concept (PoC)

```
1 cd contracts && forge test --match-contract PoC_CircuitBreakerWithdraw -vv
```

```

1 function test_POC_circuit_breaker_blocks_crash_update() public {
2     vm.expectRevert();
3     oracle.updatePrice(ETH_USD, 1200e8, T0 + 1, 5, SRC_HASH, _sign(1200e8, T0 + 1, 5));
4     (uint256 p,,) = oracle.getLatestPrice(ETH_USD);
5     assertEq(p, 2000e8, "price frozen at pre-crash level");
6 }
7
8 function test_POC_withdraw_no_freshness_while_price_stale_high() public {
9     vm.expectRevert();
10    oracle.updatePrice(ETH_USD, 1200e8, T0 + 1, 5, SRC_HASH, _sign(1200e8, T0 + 1, 5));
11
12    aToken.mint(borrower, 10 ether);
13    weth.mint(address(pool), 10 ether);
14    vm.startPrank(borrower);
15    aToken.approve(address(router), 10 ether);
16    KaskadRouter.PriceUpdate[] memory empty = new KaskadRouter.PriceUpdate[](0);
17    router.withdrawWithPrices(empty, address(weth), 10 ether);
18    vm.stopPrank();
19
20    assertEq(weth.balanceOf(borrower), 10 ether);
21    (uint256 p,,) = oracle.getLatestPrice(ETH_USD);
22    assertEq(p, 2000e8);
23 }

```

2.3.0 Mitigation

- Allow larger downward moves when a quorum of sources agrees (asymmetric breaker), or a controlled multi-step crash path liquidators can complete first.
- On withdraw, require freshness for assets that remain in the user's collateral set after the exit when the action can increase protocol risk.
- Ensure the Aave sentinel blocks risk-increasing actions during the same lag window.

2.4 Serial Source Fetching Lets Timeouts Accumulate Past the Router Freshness Window

ID: M-01

Severity: **Medium**

Components: `src/sources/mod.rs`, `src/http_client.rs`, `src/main.rs`

2.4.0 Links to Root Cause

- Comment claims concurrent fetch; implementation awaits futures serially: `src/sources/mod.rs`
- HTTP client timeout 10s / connect 5s: `src/http_client.rs`
- Main loop iterates assets sequentially: `src/main.rs`
- Documented router freshness `MAX_PRICE_AGE = 60s`: `README.md`

2.4.0 Impact

Slow or blackholed upstream sources push a fetch cycle past the documented 60-second freshness window. Router operations that require fresh signed prices fail. An attacker who delays a subset of configured sources can freeze price-dependent user actions for the duration of the timeout pile-up.

2.4.0 Description

`sources::fetch_all()` documents concurrent fetching, but creates futures and then `awaits` them one by one. Rust async futures are lazy: HTTP work begins only when polled inside the sequential loop.

Each request can take up to 10 seconds. Assets are also processed sequentially in the main loop. Timeouts therefore stack within an asset across sources and across assets across the cycle. ETH/USD and BTC/USD each configure 14 sources; a fully timing-out ETH fetch alone is on the order of $14 \times 10\text{s} = 140\text{s}$, more than double the router freshness bound.

2.4.0 Proof of Concept (PoC)

Three mock sources each sleep 150 ms. True concurrency would finish near 150 ms; measured runtime $\approx 450\text{ ms}$ proves serialization.

```

1  #[tokio::test]
2  async fn fetch_all_serializes_per_source_delay() {
3      let sources: Vec<Box<dyn PriceSource>> = vec![
4          Box::new(SlowSource { name: "slow-a", delay: ms(150), price: 1.0 }),
5          Box::new(SlowSource { name: "slow-b", delay: ms(150), price: 2.0 }),
6          Box::new(SlowSource { name: "slow-c", delay: ms(150), price: 3.0 }),
7      ];
8      let started = Instant::now();
9      let out = fetch_all(&sources, &test_asset()).await;
10     let elapsed = started.elapsed();
11     assert_eq!(out.len(), 3);
12     assert!(elapsed >= Duration::from_millis(420));
13 }
```

```
1 cargo test fetch_all_serializes_per_source_delay -- --nocapture
```

2.4.0 Mitigation

- Replace the sequential await loop with real concurrency (`FuturesUnordered` / `join_all`).
- Add a per-asset / per-cycle deadline so degraded sources cannot monopolize a cycle.
- Add a regression test that fails if `fetch_all()` time grows linearly with identical delayed sources.

2.5 Relay Hardcodes MIN_SOURCES = 3 and Drops Valid Two-Source Updates

ID: M-02
Severity: **Medium**
Components: relayer/src/relay.ts, config/assets.json, KaskadPriceOracle.sol

2.5.0 Links to Root Cause

- Per-asset enclave / on-chain quorum (e.g. USDC/USDT at 2): config/assets.json, registerAssets
- Relay global constant MIN_SOURCES = 3: relayer/src/relay.ts

2.5.0 Impact

Valid enclave-signed updates for assets with quorum 2 never reach the chain through the provided relay. The on-chain cache ages while the enclave keeps producing acceptable signatures. Aave-style consumers that read the aggregator then operate on stale prices for USDC, USDT, and every other asset configured below the hardcoded floor of 3.

2.5.0 Description

The enclave and KaskadPriceOracle enforce per-asset quora. USDC/USD, USDT/USD, and other feeds use `min_sources: 2`. The TypeScript relay rejects any update with `num_sources < 3` before submitting a transaction:

```
1 const MIN_SOURCES = 3;
2 // ...
3 if (asset.latest.num_sources < MIN_SOURCES) {
4   console.warn(
5     '[Relay] ${asset.symbol}: num_sources=${asset.latest.num_sources} < ${MIN_SOURCES}, skipping'
6   );
7   asset.state = AssetState.IDLE;
8   return;
9 }
```

The comment claims this matches “enclave Data Quorum.” Enclave quorum is per-asset, not a global 3.

2.5.0 Proof of Concept (PoC)

Direct constant check against config and contract acceptance rules:

```
1 rg -n "MIN_SOURCES|min_sources" relayer/src/relay.ts config/assets.json
```

Expected: relay constant 3; USDC/USDT (and other assets) config 2. A signed update with `num_sources: 2` for such an asset is accepted by `updatePrice` when registered with `minSources = 2`, but `tryRelay` returns early and never queues a transaction.

2.5.0 Mitigation

Replace the hardcoded constant with on-chain `assetParams(assetId).minSources` (or a per-symbol map mirrored from `assets.json`). Do not apply a global floor of 3 to assets whose configured quorum is 2.

3 Appendix

3.1 PoC Artifact Index

Finding	Artifact
C-01	tests/exploitability_pocs.rs (host sealing route)
H-01	tests/poc_volume_weight_gap.rs; TIBBIR equal-fallback PoC
H-02	contracts/test/PoC_CircuitBreakerWithdraw.t.sol
M-01	Serial <code>fetch_all</code> tokio test (see finding body)
M-02	Relayer constant vs <code>config/assets.json</code> inspection